

Data Science

Lecture 10: Projections and Dimensionality Reduction

Prof. Dr. Michael Granitzer

Converted from LNKB-tex part-en-ds-chp9-visual-analytics-projections.tex. Builds on Lecture 02 (variance, covariance), Lecture 07 (curse of dimensionality) and Lecture 08 (distance matrices).

Module 1

What a Projection Preserves

Guiding Question: *Reducing dimensions must lose information. Which information do you choose to keep?*

Seeing High-Dimensional Data

A **geometric projection** maps high-dimensional data onto a low-dimensional space — usually 2-D or 3-D, so that it can be plotted and reasoned about.

Linear projections

$$\mathbf{z} = \mathbf{P}\mathbf{x}$$

A matrix $\mathbf{P}$ maps every point. Distances and angles are distorted in a predictable, global way the map is interpretable and invertible in structure.

Non-linear projections

Preserve some **local** property — neighbourhoods, or pairwise distances — without a single global matrix. More faithful visually, harder to interpret.

The central question of the chapter: how do we find the *best* projection — and by which criterion is one projection better than another?

Guiding question (P1). Every method that follows answers it differently: random projection says "any projection, with a guarantee", PCA says "the one keeping most variance", MDS says "the one preserving distances". Naming the criterion is naming the method.

What Is Necessarily Lost

Reducing dimensions **must** lose information — the question is only which.

Variance

PCA keeps as much of it as a linear map can.

Distances

MDS keeps pairwise distances as faithfully as possible.

Guarantees

Random projection keeps distances within a provable error, cheaply.

A projection is defined by **what it chooses to preserve**. There is no best-in-general projection, only a best one for a stated criterion.

The Surprising Result

Johnson–Lindenstrauss lemma. For any $0 < \epsilon < 1$ and n points in any dimension, a projection onto

$$d = O\left(\frac{\log n}{\epsilon^2}\right)$$

dimensions exists that preserves all pairwise distances to within a factor $1 \pm \epsilon$.

The target dimension depends on the **number of points** and on the accuracy wanted — **not on the original dimensionality**. A million-dimensional dataset and a thousand-dimensional one need the same target dimension.

Speaker notes

This is genuinely counterintuitive and worth pausing on. It says high-dimensional data is "thinner" than it looks: n points span at most $n-1$ dimensions, and approximate distances need far fewer.



Construction

1. Draw a $d \times p$ matrix $\mathbf{R}$ with entries sampled independently (Gaussian, or ± 1 with equal probability).
2. Project: $\mathbf{z} = \frac{1}{\sqrt{d}} \mathbf{R} \mathbf{x}$.

Properties

No training, no data pass — the matrix does not depend on the data at all.
Extremely fast, trivially applied to new points, and it comes with a bound.

Limits

The axes are meaningless, so the result is not interpretable. It preserves distances, not structure, and works best when d is not too small.

Random projection preserves distances **with a guarantee and without looking at the**

Module 2

Principal Component Analysis

***Guiding Question:** PCA keeps the directions of largest variance. When is variance the wrong thing to keep?*

Which Direction Matters?

Consider a cloud of points that is stretched along one diagonal. Projecting onto the **short** axis collapses everything into a blob projecting onto the **long** axis keeps the points apart.

PCA's criterion: keep the directions along which the data **varies most**. Variance is treated as information directions with little variance are treated as noise.

1st component

The direction of maximum variance.

2nd component

The direction of maximum remaining variance, **orthogonal** to the first.

and so on

Each component orthogonal to all previous ones, each capturing less variance.

Speaker notes

Intuition first (P4). Ask whether "most variance" always means "most interesting" — it does not: for classification, the discriminating direction may have small variance. That caveat returns in the drawbacks slide.



What the Components Are

The components are ordered, so keeping the first m gives the **best possible m -dimensional linear approximation** in the least-squares sense.

Equivalently: PCA finds the linear subspace minimising the total squared distance from the points to their projections. Maximising retained variance and minimising reconstruction error are the **same** criterion.

PCA orders directions by **variance** and keeps the top ones. Its assumption — that variance is information — is what makes it powerful and what makes it fail.

Covariance Is the Object to Diagonalise

For centred data $\mathbf{X}$, the **covariance matrix**

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$$

holds the variance of each attribute on its diagonal and the covariance of each pair off it.

Large diagonal entry

That attribute varies a lot — potentially informative.

Large off-diagonal entry

Two attributes are **redundant**: one partly predicts the other.

The goal: find new axes in which the covariance matrix is **diagonal** — no redundancy left between components — and order them by variance.

Speaker notes

Framing PCA as "remove redundancy, keep variation" makes the covariance matrix the natural object. It also connects directly to the joint-distribution unit from Lecture 02, where covariance was introduced.



The Solution

The directions that diagonalise **C** are its **eigenvectors** the variance captured by each is the corresponding **eigenvalue**.

Sorting eigenvectors by descending eigenvalue gives the principal components, and the projection matrix is the top m of them.

Orthogonal

Eigenvectors of a symmetric matrix are mutually orthogonal — the components are uncorrelated by construction.

Ordered

$\lambda_1 \geq \lambda_2 \geq \dots$ ranks the directions by variance.

Explained variance

$\lambda_i / \sum_j \lambda_j$ is the share of total variance in component i .

The explained-variance ratio is the practical output students will actually use — it is what a scree plot shows and how "how many components?" gets answered.

The Procedure

1. **Centre** the data (and usually **standardise** it).
2. Compute the covariance matrix $\mathbf{C}$.
3. Compute its eigenvectors and eigenvalues.
4. Sort by descending eigenvalue keep the top m .
5. Project: $\mathbf{Z} = \mathbf{X}\mathbf{W}_m$.

Step 1 is not optional. Without standardisation, an attribute measured in metres dominates one measured in kilometres purely through its **unit** — PCA would report the choice of unit as the principal direction.

Also known as the discrete Karhunen–Loève transform. The standardisation trap is the single most common practical error and worth stating as a rule: always ask what the units are before running PCA.

Choosing the Number of Components

A **scree plot** shows eigenvalues in descending order; the usual reading is the same elbow heuristic as for choosing k in clustering, or a cumulative explained-variance threshold (80%, 95%).

Benefits

Removes correlated and noisy directions
speeds up downstream methods makes 2-D visualisation possible components are uncorrelated by construction.

Drawbacks

Components are **linear combinations of all attributes**, so interpretability is lost. Only **linear** structure is captured. And variance is assumed to equal importance — which for classification can be false.

Mini-Exercise

You run PCA on the iris dataset and the first two components explain 96% of the variance.

1. What does that number mean?
2. Does it mean the two components are good features for classifying species?
3. When could a **low-variance** direction be the important one?

Mini-Exercise — Solution

PCA is centre $\rightarrow$ covariance $\rightarrow$ eigendecomposition $\rightarrow$ keep the top components. It optimises **reconstruction**, not discrimination, and it must be standardised first.

Mini-Exercise — Solution

1. A 2-D linear approximation reconstructs the data with only 4% of the total squared variation lost.

2. Not necessarily. PCA is **unsupervised** — it never saw the labels, and optimises reconstruction, not separability.

3. Whenever the classes differ along a direction with small spread — a thin but decisive offset. Discriminant analysis targets exactly that instead.

PCA is centre → covariance → eigendecomposition → keep the top components. It optimises **reconstruction**, not discrimination, and it must be standardised first.

Points and Variables in One Picture

A **biplot** overlays two things on the same axes:

The objects

Each data point, plotted by its scores on the first two components — an ordinary scatterplot of the projection.

The variables

Each original attribute drawn as an **arrow**: its loading on the two components. Direction shows what the attribute contributes to, length shows how strongly.

Arrows together

Those attributes are positively correlated.

Arrows opposed

Negatively correlated.

Arrows at 90°

Roughly uncorrelated.

A biplot is how you recover interpretability after PCA has destroyed it: the arrows say what



the anonymous components *mean* in terms of the original attributes.

The biplot answers "what do these axes mean?" by drawing the original attributes into the projected space — points and variables read together.

Worth demonstrating on any two-component PCA: students can immediately name PC1 ("overall size") and PC2 ("shape") from the arrow pattern, which no eigenvalue table conveys.

Module 3

Beyond Variance

Guiding Question: *Some objects have no coordinates — only similarities. Can they still be mapped?*

The Problem With Matching Words

A document is represented as a vector of term counts — the **term–document matrix**. Similarity is then the overlap of vocabulary, which fails twice:

Synonymy

"car" and "automobile" never match, though the documents are about the same thing.

Polysemy

"bank" matches across finance and rivers, though the documents are unrelated.

The vocabulary is the wrong space. We want a space of **concepts**, in which related terms lie close together.

The two failures are the motivation for every embedding method that followed, right up to modern language models. Say so — students recognise the problem statement from tools they use.

The Approach

Apply a **singular value decomposition** to the term–document matrix $\mathbf{A}$:

$$\mathbf{A} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$$

and keep the k largest singular values. Documents and terms are both represented in the resulting k -dimensional **latent space**.

Dimension reduction

Thousands of terms collapse to a few hundred latent dimensions.

Synonyms merge

Terms that co-occur with the same contexts end up close together.

Noise drops

Rare, idiosyncratic usage lives in the discarded small singular values.

SVD is PCA's underlying machinery applied to a rectangular matrix — the same "keep the strongest directions" idea, on term–document data.

LSA projects documents and terms into a shared **latent space** via SVD, so that similarity survives synonymy. It is dimensionality reduction used as a representation, not just as compression.

Speaker notes

Connect forward honestly: modern embeddings solve the same problem with learned, non-linear, context-sensitive representations — but the objective LSA states is still recognisable in them.



A Different Criterion

PCA needs coordinates and preserves **variance**. MDS needs only a **distance matrix** and preserves **the distances themselves**.

Given pairwise distances d_{ij} , find low-dimensional points $\mathbf{z}_1, \dots, \mathbf{z}_n$ minimising the **stress**
$$S = \sum_{i < j} (\|\mathbf{z}_i - \mathbf{z}_j\| - d_{ij})^2$$

This is the projection method for objects that have **no coordinates at all** — only a similarity: documents, molecules, survey respondents, perceived similarity ratings.

Speaker notes

The distance-matrix-only requirement is the same property that made HAC applicable to non-vector data. Point at that parallel — it is the second time this course uses "all I need is a similarity".



Classical MDS and Its Relatives

Classical (metric) MDS

Solved in closed form by an eigendecomposition of the (double-centred) squared distance matrix. With Euclidean distances it is **equivalent to PCA**.

Non-metric MDS

Preserves only the **rank order** of distances. Suited to ordinal similarity data — "A is more like B than like C" — where the numbers are not meaningful.

Reading the Result

Stress tells you the cost

A 2-D map of genuinely high-dimensional structure has high stress: the picture is a compromise, and the number says how big a one.

Axes mean nothing

Unlike PCA components, MDS axes have no interpretation — only **relative positions** carry information. Rotations and reflections are equivalent.

MDS preserves **distances** rather than variance, works from a similarity matrix alone, and produces a map whose axes are meaningless but whose **neighbourhoods are faithful** — as far as the stress allows.

The "axes mean nothing" warning matters because students read MDS plots like scatterplots. Insist that only the pattern of proximity may be interpreted.

